



STEP-1: EXPLORATORY DATA ANALYSIS

In [1]: *# STEP 1: IMPORTING REQUIRED LIBRARIES*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]: *# STEP 2: SET DISPLAY OPTION*

```
pd.set_option("display.max_columns", None)
pd.set_option('display.max_rows', None)
```

In [12]: *# STEP 3: LOADING THE DATASET*

```
df=pd.read_csv("WineQT.csv")
```

In [13]: *# SHOWING 5 ROWS*

```
df.head(50)
```

Out[13]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH
0	7.4	0.700	0.00	1.90	0.076	11.0	34.0	0.9978	3.51
1	7.8	0.880	0.00	2.60	0.098	25.0	67.0	0.9968	3.20
2	7.8	0.760	0.04	2.30	0.092	15.0	54.0	0.9970	3.26
3	11.2	0.280	0.56	1.90	0.075	17.0	60.0	0.9980	3.16
4	7.4	0.700	0.00	1.90	0.076	11.0	34.0	0.9978	3.51
5	7.4	0.660	0.00	1.80	0.075	13.0	40.0	0.9978	3.51
6	7.9	0.600	0.06	1.60	0.069	15.0	59.0	0.9964	3.30
7	7.3	0.650	0.00	1.20	0.065	15.0	21.0	0.9946	3.39
8	7.8	0.580	0.02	2.00	0.073	9.0	18.0	0.9968	3.36
9	6.7	0.580	0.08	1.80	0.097	15.0	65.0	0.9959	3.28
10	5.6	0.615	0.00	1.60	0.089	16.0	59.0	0.9943	3.58
11	7.8	0.610	0.29	1.60	0.114	9.0	29.0	0.9974	3.26
12	8.5	0.280	0.56	1.80	0.092	35.0	103.0	0.9969	3.30
13	7.9	0.320	0.51	1.80	0.341	17.0	56.0	0.9969	3.04
14	7.6	0.390	0.31	2.30	0.082	23.0	71.0	0.9982	3.52
15	7.9	0.430	0.21	1.60	0.106	10.0	37.0	0.9966	3.17
16	8.5	0.490	0.11	2.30	0.084	9.0	67.0	0.9968	3.17
17	6.9	0.400	0.14	2.40	0.085	21.0	40.0	0.9968	3.43
18	6.3	0.390	0.16	1.40	0.080	11.0	23.0	0.9955	3.34
19	7.6	0.410	0.24	1.80	0.080	4.0	11.0	0.9962	3.28
20	7.1	0.710	0.00	1.90	0.080	14.0	35.0	0.9972	3.47
21	7.8	0.645	0.00	2.00	0.082	8.0	16.0	0.9964	3.38
22	6.7	0.675	0.07	2.40	0.089	17.0	82.0	0.9958	3.35
23	8.3	0.655	0.12	2.30	0.083	15.0	113.0	0.9966	3.17
24	5.2	0.320	0.25	1.80	0.103	13.0	50.0	0.9957	3.38
25	7.8	0.645	0.00	5.50	0.086	5.0	18.0	0.9986	3.40
26	7.8	0.600	0.14	2.40	0.086	3.0	15.0	0.9975	3.42
27	8.1	0.380	0.28	2.10	0.066	13.0	30.0	0.9968	3.23
28	7.3	0.450	0.36	5.90	0.074	12.0	87.0	0.9978	3.33
29	8.8	0.610	0.30	2.80	0.088	17.0	46.0	0.9976	3.26

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH
30	7.5	0.490	0.20	2.60	0.332	8.0	14.0	0.9968	3.21
31	8.1	0.660	0.22	2.20	0.069	9.0	23.0	0.9968	3.30
32	4.6	0.520	0.15	2.10	0.054	8.0	65.0	0.9934	3.90
33	7.7	0.935	0.43	2.20	0.114	22.0	114.0	0.9970	3.25
34	8.8	0.660	0.26	1.70	0.074	4.0	23.0	0.9971	3.15
35	6.6	0.520	0.04	2.20	0.069	8.0	15.0	0.9956	3.40
36	6.6	0.500	0.04	2.10	0.068	6.0	14.0	0.9955	3.39
37	8.6	0.380	0.36	3.00	0.081	30.0	119.0	0.9970	3.20
38	7.6	0.510	0.15	2.80	0.110	33.0	73.0	0.9955	3.17
39	10.2	0.420	0.57	3.40	0.070	4.0	10.0	0.9971	3.04
40	7.8	0.590	0.18	2.30	0.076	17.0	54.0	0.9975	3.43
41	7.3	0.390	0.31	2.40	0.074	9.0	46.0	0.9962	3.41
42	8.8	0.400	0.40	2.20	0.079	19.0	52.0	0.9980	3.44
43	7.7	0.690	0.49	1.80	0.115	20.0	112.0	0.9968	3.21
44	7.0	0.735	0.05	2.00	0.081	13.0	54.0	0.9966	3.39
45	7.2	0.725	0.05	4.65	0.086	4.0	11.0	0.9962	3.41
46	7.2	0.725	0.05	4.65	0.086	4.0	11.0	0.9962	3.41
47	6.6	0.705	0.07	1.60	0.076	6.0	15.0	0.9962	3.44
48	8.0	0.705	0.05	1.90	0.074	8.0	19.0	0.9962	3.34
49	7.7	0.690	0.22	1.90	0.084	18.0	94.0	0.9961	3.31

In [10]: `# STEP 4: BASIC DATA CHECK`

```
print("SHAPE:\n", df.shape)
```

SHAPE:
(1143, 13)

In [11]: `print("\nINFO:\n")
print(df.info())`

INFO:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   fixed acidity          1143 non-null   float64
1   volatile acidity       1143 non-null   float64
2   citric acid            1143 non-null   float64
3   residual sugar         1143 non-null   float64
4   chlorides              1143 non-null   float64
5   free sulfur dioxide    1143 non-null   float64
6   total sulfur dioxide   1143 non-null   float64
7   density                1143 non-null   float64
8   pH                    1143 non-null   float64
9   sulphates              1143 non-null   float64
10  alcohol                1143 non-null   float64
11  quality                1143 non-null   int64
12  Id                     1143 non-null   int64
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
None
```

```
In [14]: print("\nDESCRIBE:\n")
         print(df.describe(include='all'))
```

DESCRIBE:

	fixed acidity	volatile acidity	citric acid	residual sugar \
count	1143.000000	1143.000000	1143.000000	1143.000000
mean	8.311111	0.531339	0.268364	2.532152
std	1.747595	0.179633	0.196686	1.355917
min	4.600000	0.120000	0.000000	0.900000
25%	7.100000	0.392500	0.090000	1.900000
50%	7.900000	0.520000	0.250000	2.200000
75%	9.100000	0.640000	0.420000	2.600000
max	15.900000	1.580000	1.000000	15.500000

	chlorides	free sulfur dioxide	total sulfur dioxide	density \
count	1143.000000	1143.000000	1143.000000	1143.000000
mean	0.086933	15.615486	45.914698	0.996730
std	0.047267	10.250486	32.782130	0.001925
min	0.012000	1.000000	6.000000	0.990070
25%	0.070000	7.000000	21.000000	0.995570
50%	0.079000	13.000000	37.000000	0.996680
75%	0.090000	21.000000	61.000000	0.997845
max	0.611000	68.000000	289.000000	1.003690

	pH	sulphates	alcohol	quality	Id
count	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000
mean	3.311015	0.657708	10.442111	5.657043	804.969379
std	0.156664	0.170399	1.082196	0.805824	463.997116
min	2.740000	0.330000	8.400000	3.000000	0.000000
25%	3.205000	0.550000	9.500000	5.000000	411.000000
50%	3.310000	0.620000	10.200000	6.000000	794.000000
75%	3.400000	0.730000	11.100000	6.000000	1209.500000
max	4.010000	2.000000	14.900000	8.000000	1597.000000

```
In [15]: print("\nCOLUMNS:\n")
print(df.columns)
```

COLUMNS:

```
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar',
      'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density',
      'pH', 'sulphates', 'alcohol', 'quality', 'Id'],
      dtype='object')
```

```
In [16]: # STEP 5: CLEAN COLUMN NAMES
df.columns = df.columns.str.strip().str.upper().str.replace(" ", "_")
df.columns
```

```
Out[16]: Index(['FIXED_ACIDITY', 'VOLATILE_ACIDITY', 'CITRIC_ACID', 'RESIDUAL_SUGAR',
      'CHLORIDES', 'FREE_SULFUR_DIOXIDE', 'TOTAL_SULFUR_DIOXIDE', 'DENSITY',
      'PH', 'SULPHATES', 'ALCOHOL', 'QUALITY', 'ID'],
      dtype='object')
```

```
In [17]: # STEP 6: CHECK MISSING VALUES
df.isnull().sum()      # null values
```

```
Out[17]:
```

	0
FIXED_ACIDITY	0
VOLATILE_ACIDITY	0
CITRIC_ACID	0
RESIDUAL_SUGAR	0
CHLORIDES	0
FREE_SULFUR_DIOXIDE	0
TOTAL_SULFUR_DIOXIDE	0
DENSITY	0
PH	0
SULPHATES	0
ALCOHOL	0
QUALITY	0
ID	0

dtype: int64

```
In [18]: df.notnull().sum()    # not-null values
```

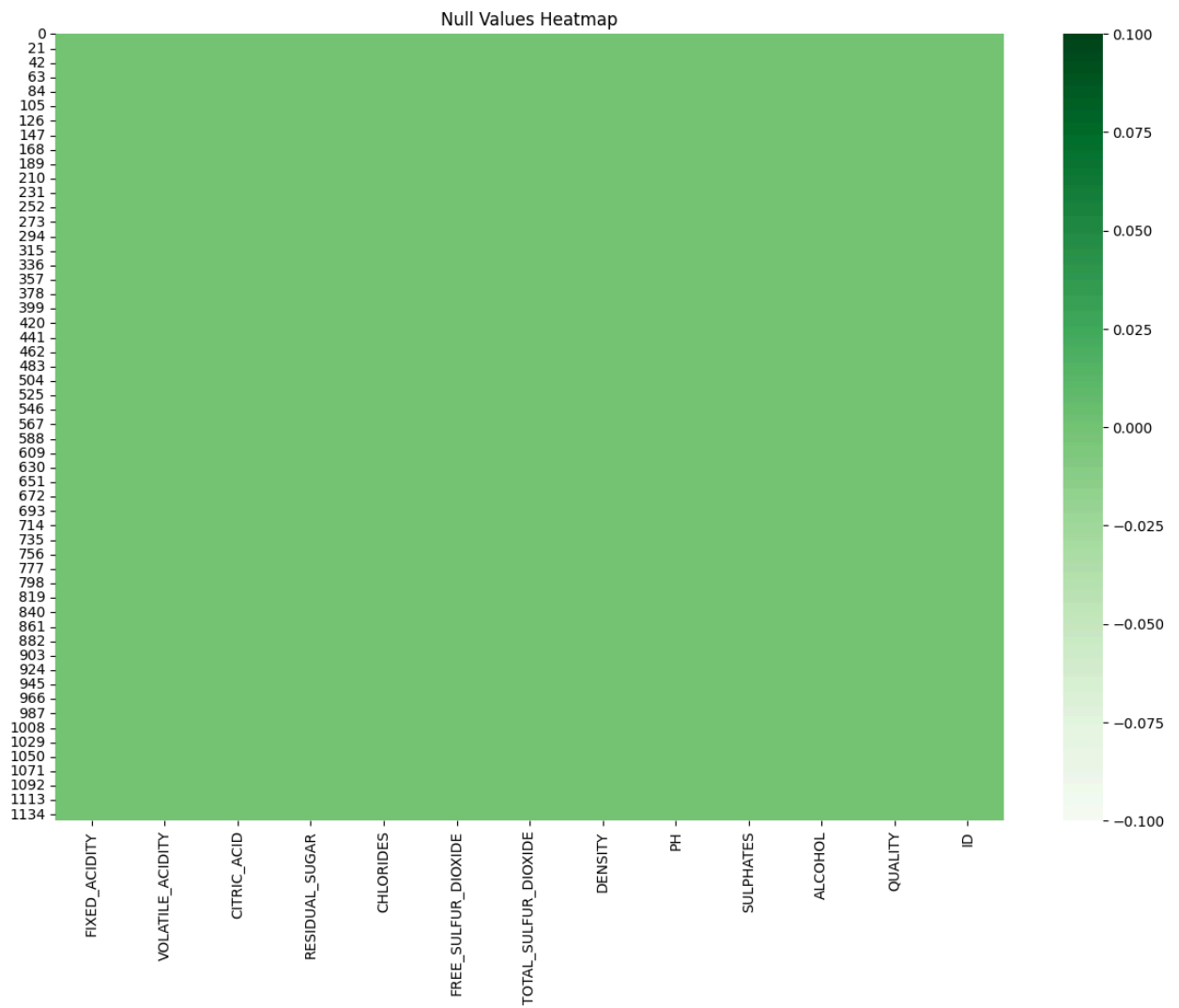
Out[18]:

0

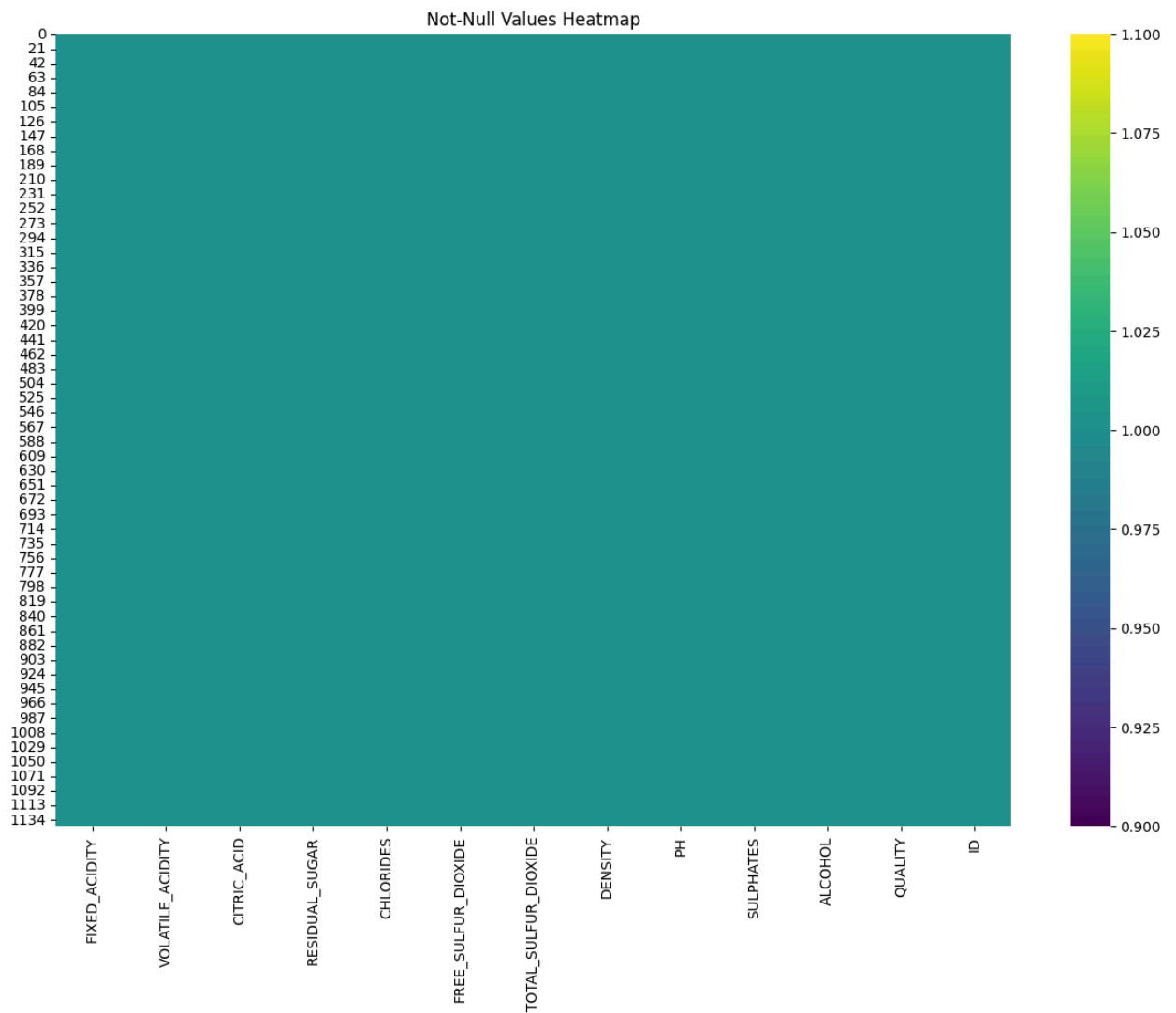
FIXED_ACIDITY	1143
VOLATILE_ACIDITY	1143
CITRIC_ACID	1143
RESIDUAL_SUGAR	1143
CHLORIDES	1143
FREE_SULFUR_DIOXIDE	1143
TOTAL_SULFUR_DIOXIDE	1143
DENSITY	1143
PH	1143
SULPHATES	1143
ALCOHOL	1143
QUALITY	1143
ID	1143

dtype: int64

```
In [21]: # STEP 7: HEATMAP OF NULL VALUES
plt.figure(figsize=(15, 10))
sns.heatmap(df.isnull(), cmap="Greens", cbar=True)
plt.title("Null Values Heatmap")
plt.show()
```



```
In [22]: plt.figure(figsize=(15, 10))
sns.heatmap(df.notnull(), cbar=True, cmap="viridis")
plt.title("Not-Null Values Heatmap")
plt.show()
```

```
In [23]: # STEP 8: CHECK DUPLICATES
df.duplicated().sum()
```

```
Out[23]: np.int64(0)
```

```
In [24]: # Drop duplicates
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

```
Shape after removing duplicates: (1143, 13)
```

```
In [25]: # STEP 9: CHECK & FIX DATA TYPES
df.dtypes
```

Out[25]:

	0
FIXED_ACIDITY	float64
VOLATILE_ACIDITY	float64
CITRIC_ACID	float64
RESIDUAL_SUGAR	float64
CHLORIDES	float64
FREE_SULFUR_DIOXIDE	float64
TOTAL_SULFUR_DIOXIDE	float64
DENSITY	float64
PH	float64
SULPHATES	float64
ALCOHOL	float64
QUALITY	int64
ID	int64

dtype: object

```
In [26]: # DROPPING NULL VALUES
df=df.dropna()
```

```
In [27]: # RECHECK
df.head()
```

```
Out[27]:
```

	FIXED_ACIDITY	VOLATILE_ACIDITY	CITRIC_ACID	RESIDUAL_SUGAR	CHLORIDE
0	7.4	0.70	0.00	1.9	0.07
1	7.8	0.88	0.00	2.6	0.09
2	7.8	0.76	0.04	2.3	0.09
3	11.2	0.28	0.56	1.9	0.07
4	7.4	0.70	0.00	1.9	0.07

```
In [28]: # RECHECK
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   FIXED_ACIDITY          1143 non-null   float64
1   VOLATILE_ACIDITY        1143 non-null   float64
2   CITRIC_ACID             1143 non-null   float64
3   RESIDUAL_SUGAR          1143 non-null   float64
4   CHLORIDES               1143 non-null   float64
5   FREE_SULFUR_DIOXIDE     1143 non-null   float64
6   TOTAL_SULFUR_DIOXIDE    1143 non-null   float64
7   DENSITY                 1143 non-null   float64
8   PH                      1143 non-null   float64
9   SULPHATES               1143 non-null   float64
10  ALCOHOL                 1143 non-null   float64
11  QUALITY                  1143 non-null   int64
12  ID                      1143 non-null   int64
dtypes: float64(11), int64(2)
memory usage: 116.2 KB

```

```

In [29]: # RECHECK
df.isnull().sum()/len(df)*100    # null values

```

```

Out[29]:
0
FIXED_ACIDITY  0.0
VOLATILE_ACIDITY  0.0
CITRIC_ACID  0.0
RESIDUAL_SUGAR  0.0
CHLORIDES  0.0
FREE_SULFUR_DIOXIDE  0.0
TOTAL_SULFUR_DIOXIDE  0.0
DENSITY  0.0
PH  0.0
SULPHATES  0.0
ALCOHOL  0.0
QUALITY  0.0
ID  0.0

```

dtype: float64

```

In [30]: df.columns

```

```
Out[30]: Index(['FIXED_ACIDITY', 'VOLATILE_ACIDITY', 'CITRIC_ACID', 'RESIDUAL_SUGAR',  
              'CHLORIDES', 'FREE_SULFUR_DIOXIDE', 'TOTAL_SULFUR_DIOXIDE', 'DENSITY',  
              'PH', 'SULPHATES', 'ALCOHOL', 'QUALITY', 'ID'],  
              dtype='object')
```

```
In [36]: # STEP 11: VISUALIZATIONS  
# HISTOGRAM – ALCOHOL  
plt.figure(figsize=(15, 8))  
sns.histplot(df["ALCOHOL"], color="teal")  
plt.xlabel("ALCOHOL")  
plt.ylabel("FREQUENCY")  
plt.title("Histogram of ALCOHOL")  
plt.show
```

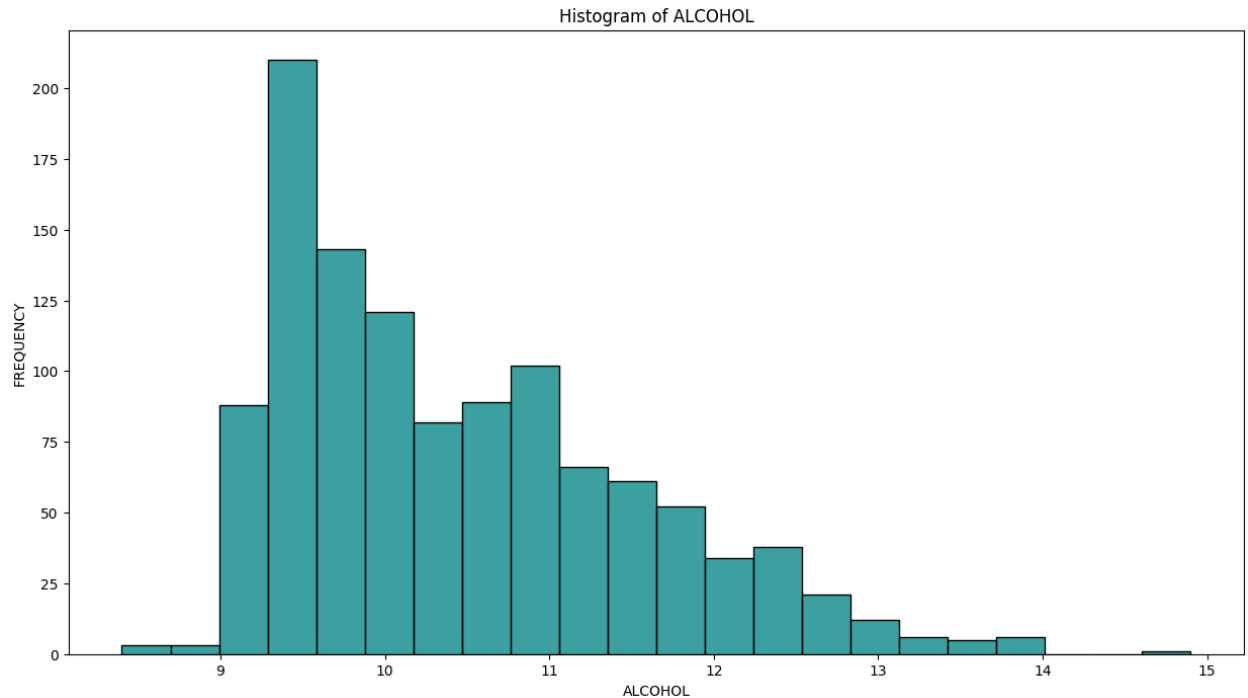
Out[36]: **matplotlib.pyplot.show**
def show(*args, **kwargs) -> None

Display all open figures.

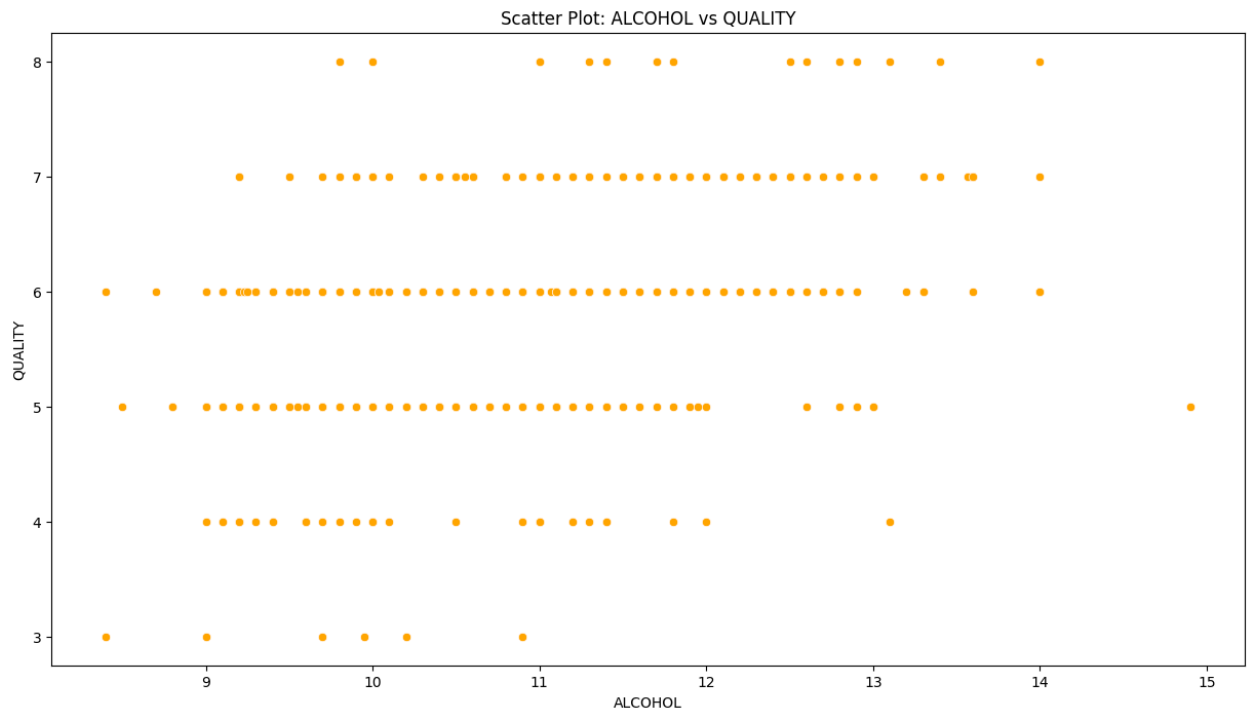
Parameters

block : bool, optional

The jupyter backends (activated via ``%matplotlib inline``)

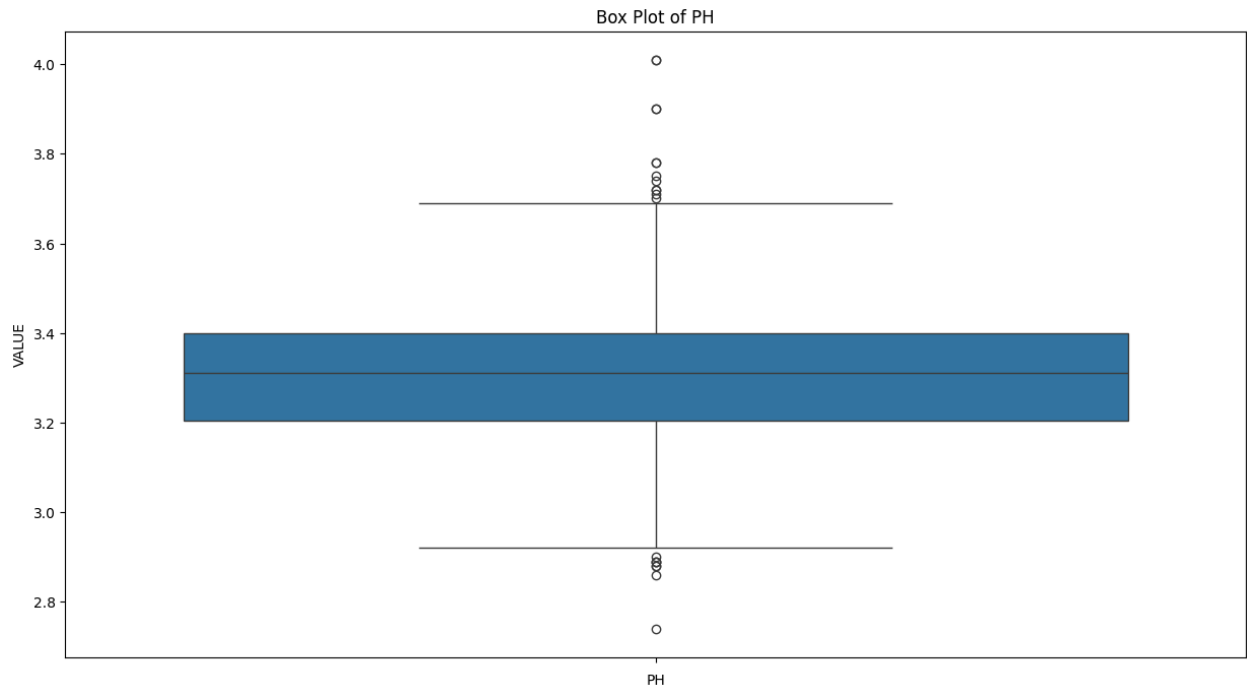


```
In [37]: # SCATTER PLOT – ALCOHOL vs QUALITY
plt.figure(figsize=(15, 8))
sns.scatterplot(x=df["ALCOHOL"], y=df["QUALITY"], color="orange")
plt.xlabel("ALCOHOL")
plt.ylabel("QUALITY")
plt.title("Scatter Plot: ALCOHOL vs QUALITY")
plt.show()
```

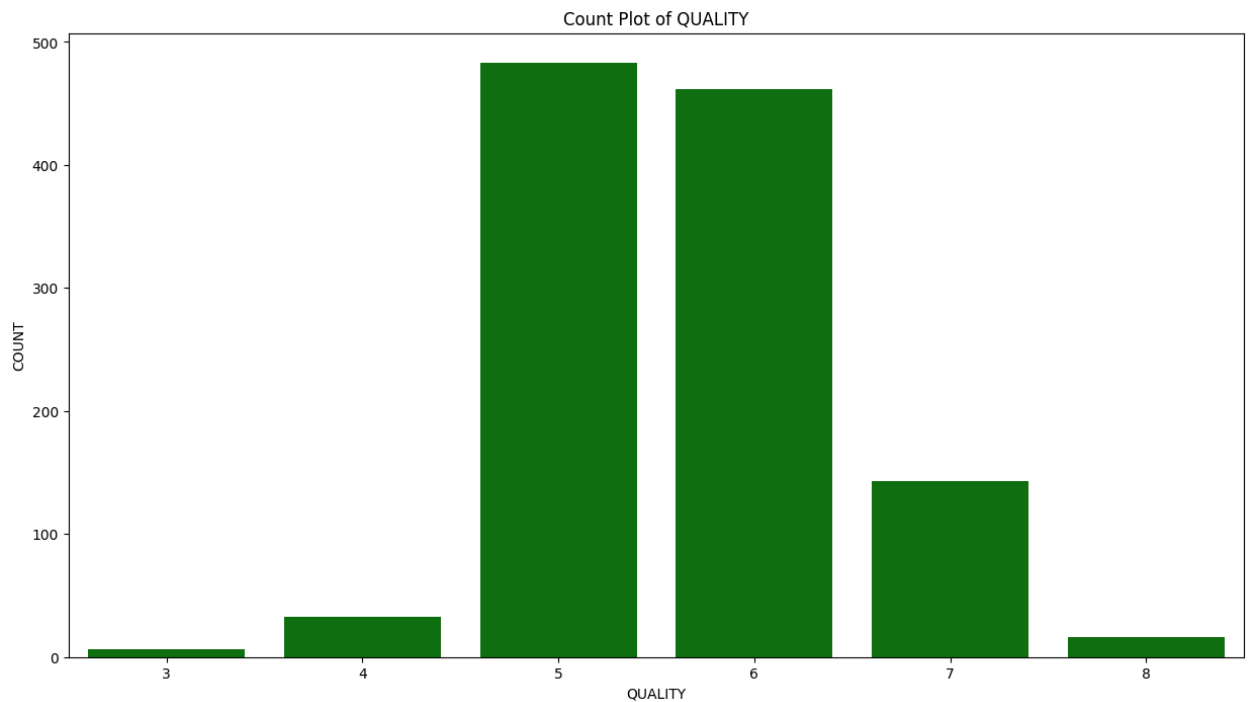


```
In [40]: # BOX PLOT – PH
```

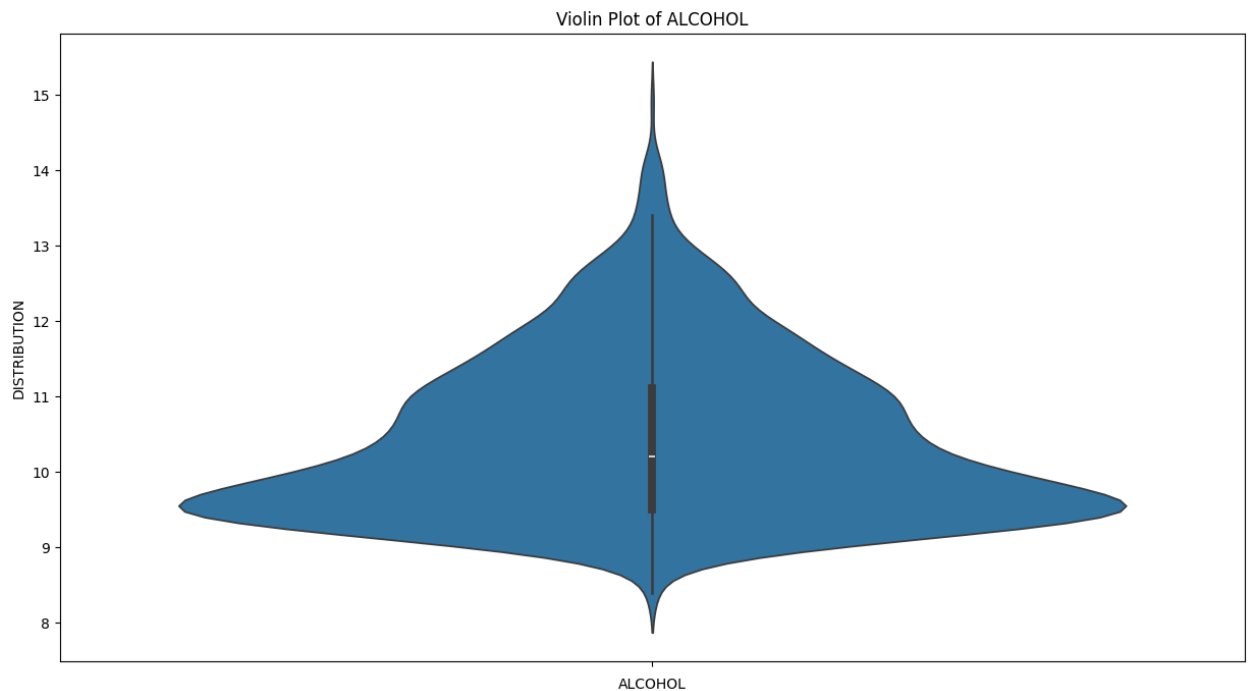
```
plt.figure(figsize=(15, 8))
sns.boxplot(y=df["PH"])
plt.xlabel("PH")
plt.ylabel("VALUE")
plt.title("Box Plot of PH")
plt.show()
```



```
In [42]: # COUNT PLOT - QUALITY
plt.figure(figsize=(15, 8))
sns.countplot(x=df["QUALITY"], color="green")
plt.xlabel("QUALITY")
plt.ylabel("COUNT")
plt.title("Count Plot of QUALITY")
plt.show()
```



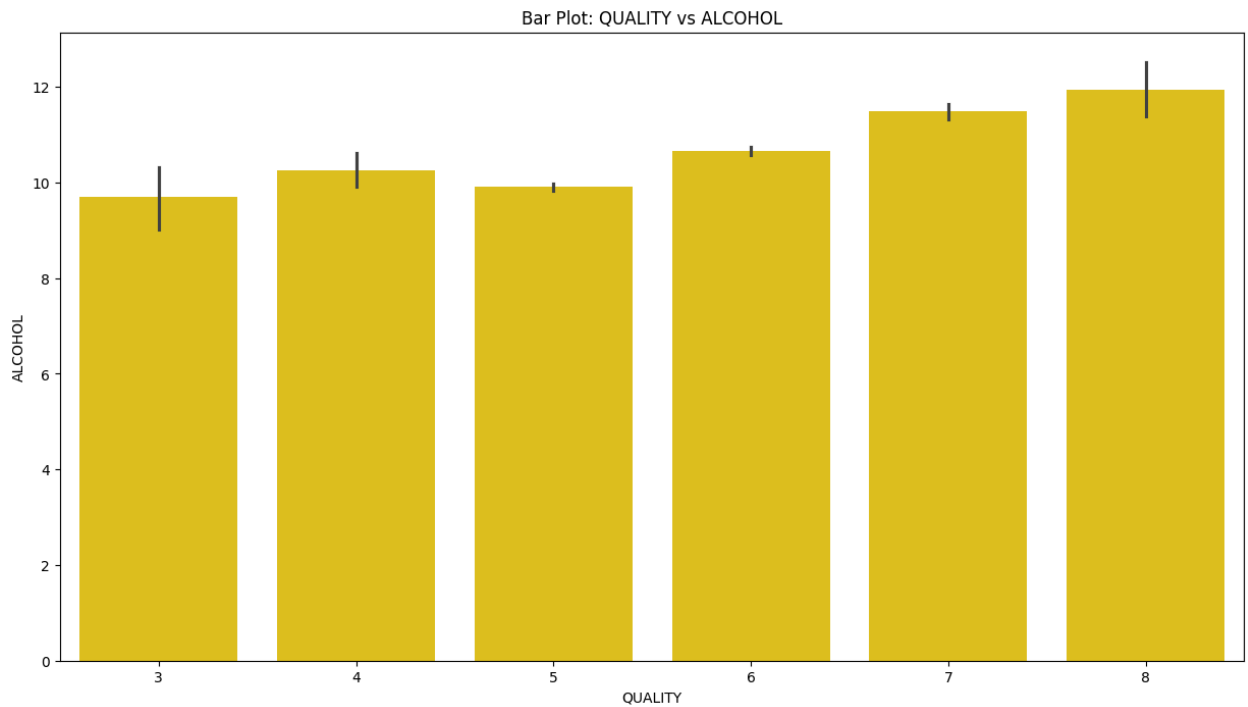
```
In [45]: # VIOLIN PLOT – ALCOHOL
plt.figure(figsize=(15, 8))
sns.violinplot(y=df["ALCOHOL"])
plt.xlabel("ALCOHOL")
plt.ylabel("DISTRIBUTION")
plt.title("Violin Plot of ALCOHOL")
plt.show()
```



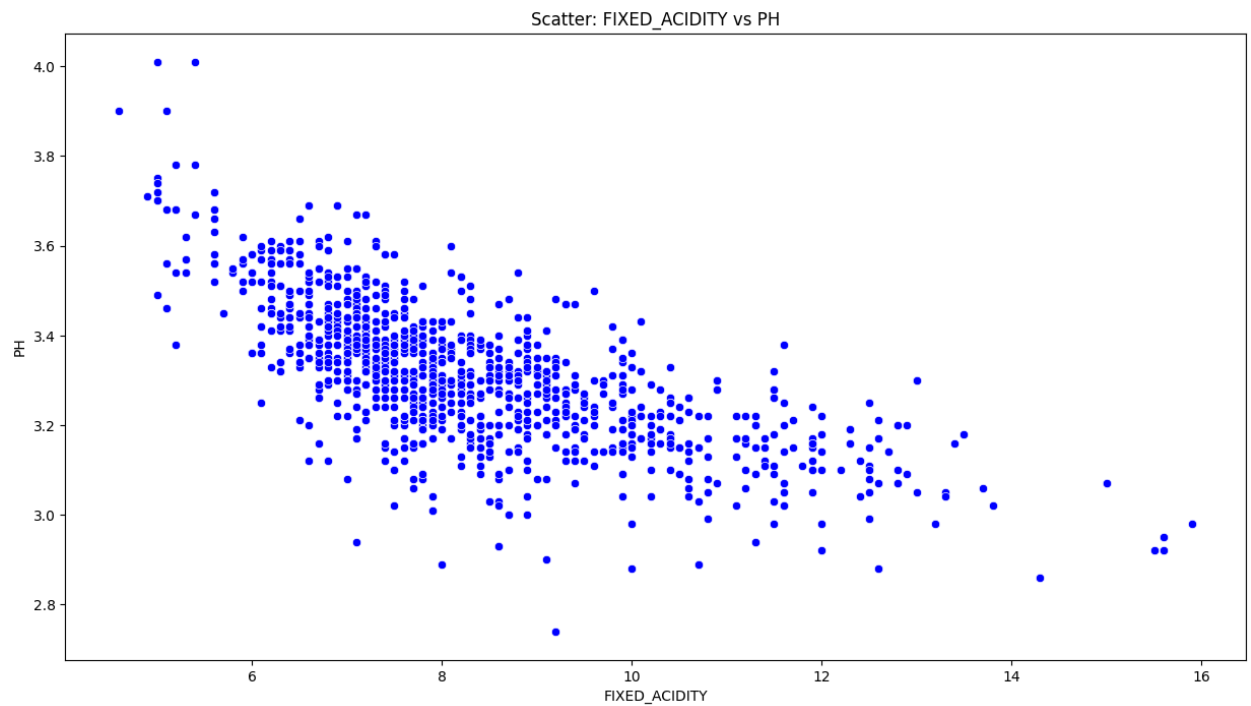
```
In [47]: # BAR PLOT – QUALITY vs ALCOHOL
plt.figure(figsize=(15, 8))
```



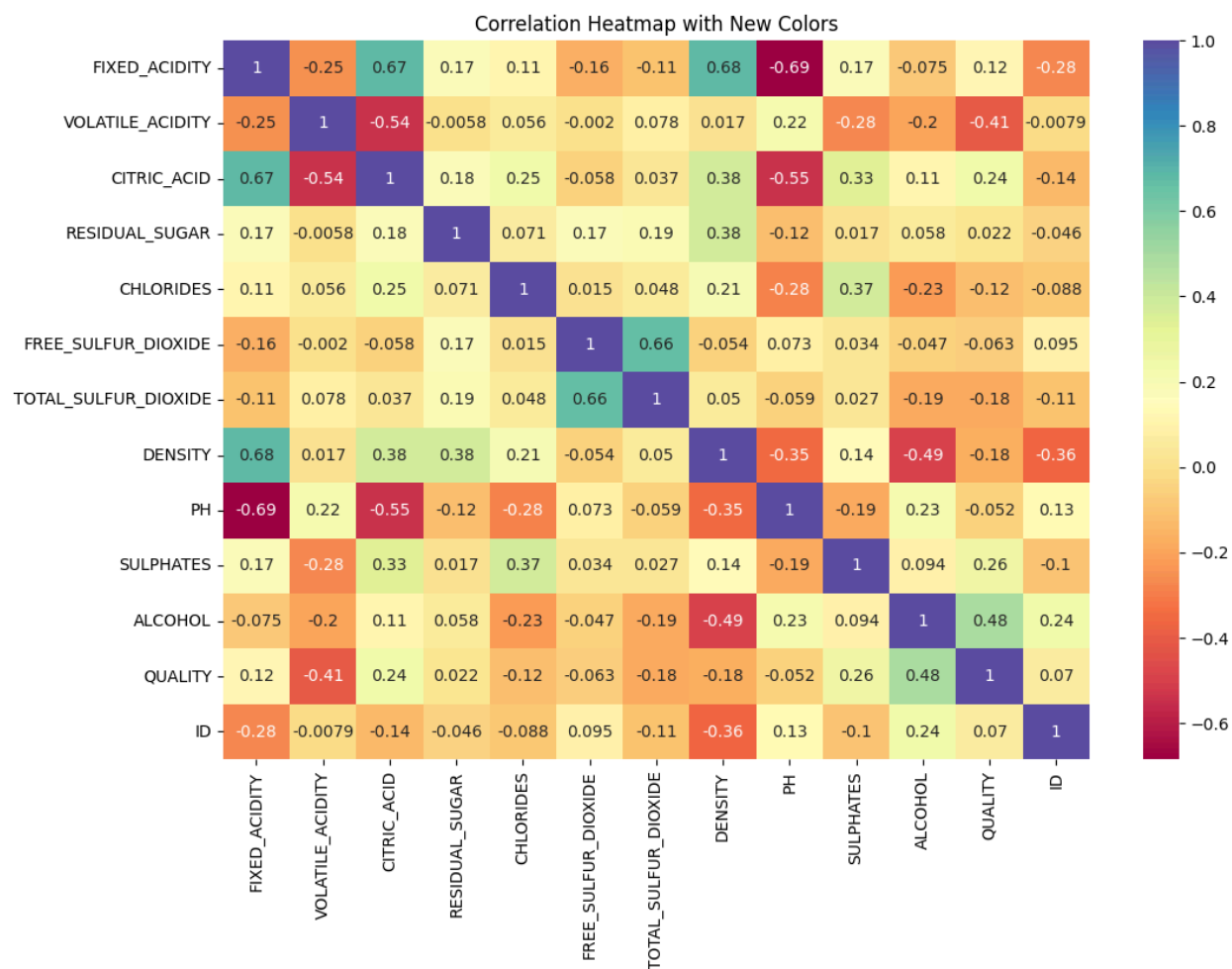
```
sns.barplot(x=df["QUALITY"], y=df["ALCOHOL"], color="gold")
plt.xlabel("QUALITY")
plt.ylabel("ALCOHOL")
plt.title("Bar Plot: QUALITY vs ALCOHOL")
plt.show()
```



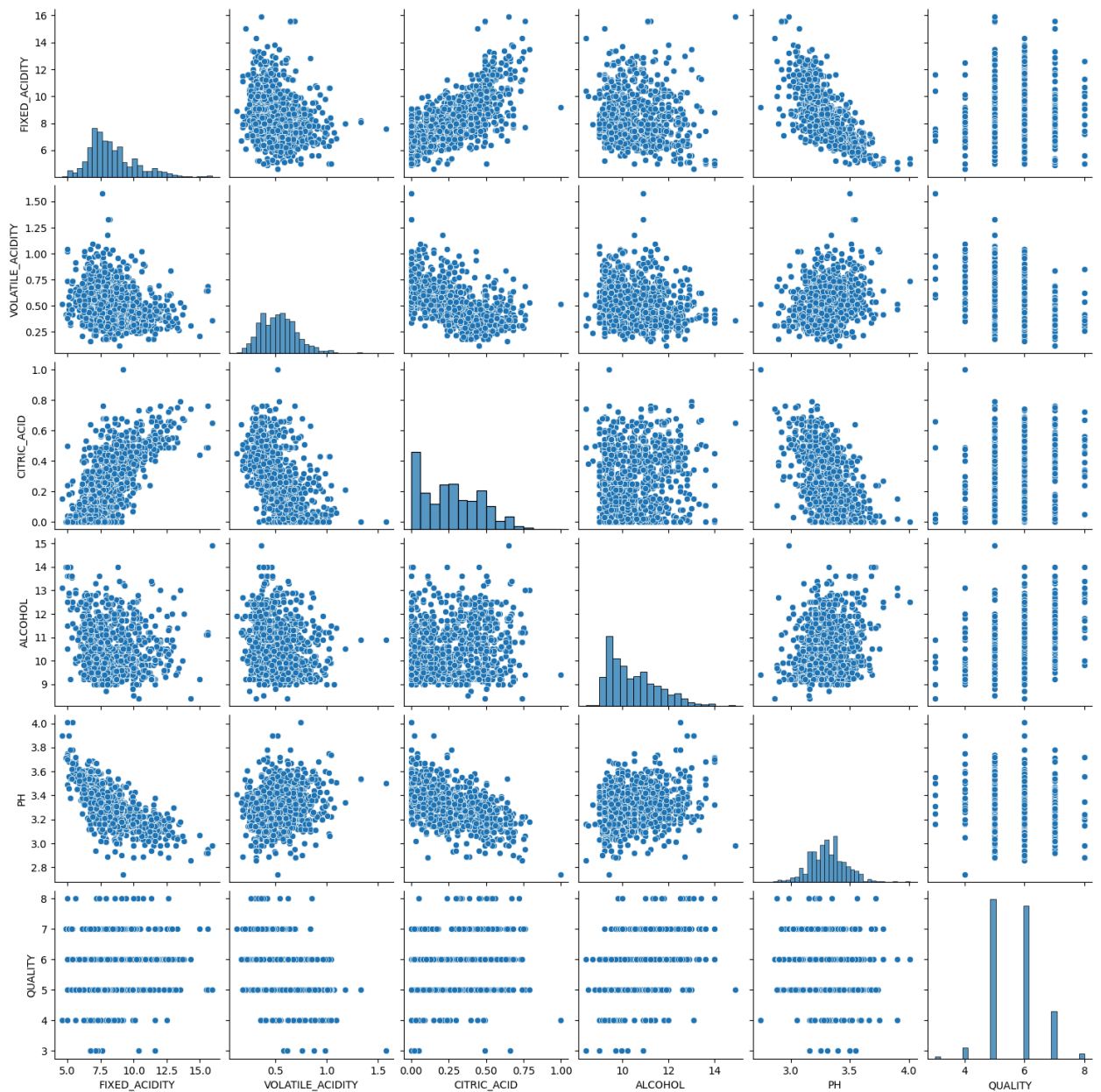
```
In [50]: # SCATTER - FIXED_ACIDITY vs PH
plt.figure(figsize=(15, 8))
sns.scatterplot(x=df["FIXED_ACIDITY"], y=df["PH"], color="blue")
plt.xlabel("FIXED_ACIDITY")
plt.ylabel("PH")
plt.title("Scatter: FIXED_ACIDITY vs PH")
plt.show()
```



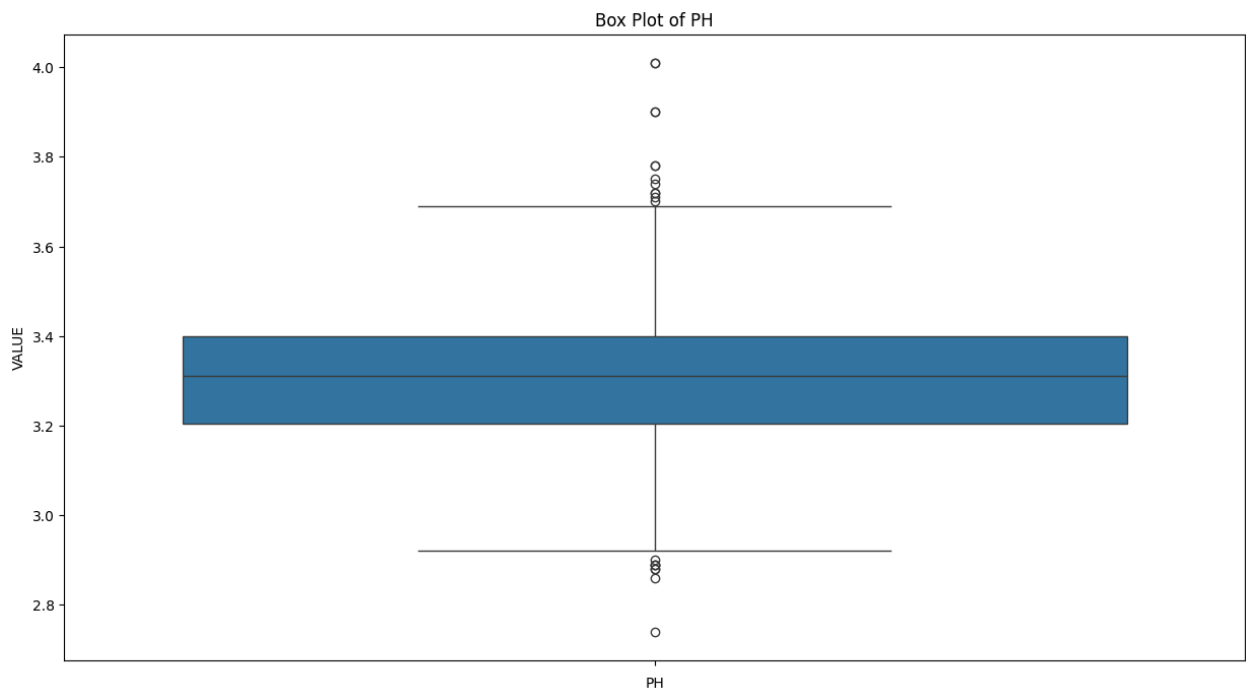
```
In [52]: # HEATMAP WITH NEW MULTI COLORS
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap="Spectral")
plt.title("Correlation Heatmap")
plt.show()
```



```
In [54]: # PAIRPLOT
sns.pairplot(
    df[[
        "FIXED_ACIDITY",
        "VOLATILE_ACIDITY",
        "CITRIC_ACID",
        "ALCOHOL",
        "PH",
        "QUALITY"
    ]])
plt.show()
```



```
In [55]: # step 12: HANDLE OUTLIERS OF BOX PLOT
# BOX PLOT – PH
plt.figure(figsize=(15, 8))
sns.boxplot(y=df["PH"])
plt.xlabel("PH")
plt.ylabel("VALUE")
plt.title("Box Plot of PH")
plt.show()
```



```
In [56]: # Function to cap outliers using IQR method
def cap_outliers(column):
    Q1 = column.quantile(0.25)    # First quartile (25%)
    Q3 = column.quantile(0.75)    # Third quartile (75%)
    IQR = Q3 - Q1                 # Interquartile range

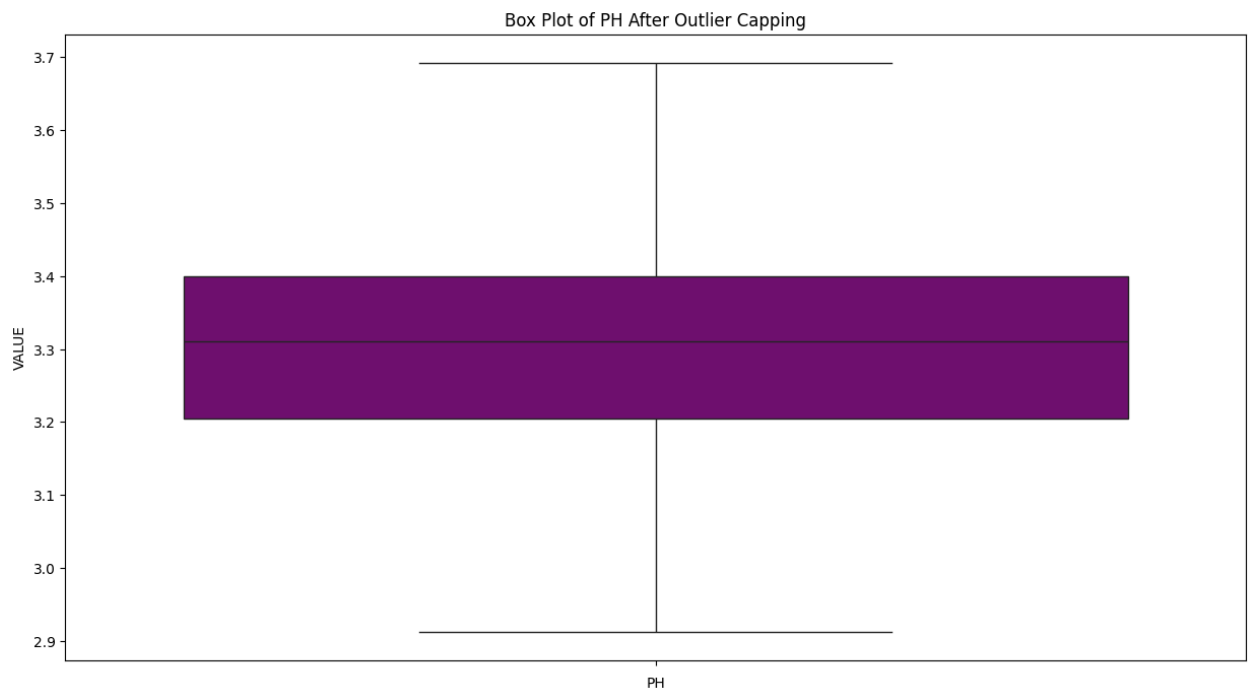
    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    # Apply capping
    column = np.where(column < lower_limit, lower_limit, column)
    column = np.where(column > upper_limit, upper_limit, column)

    return column
```

```
In [57]: # APPLY CAPPING TO PH COLUMN
df["PH"] = cap_outliers(df["PH"])
```

```
In [59]: # UPDATED BOX PLOT AFTER OUTLIER CAPPING
plt.figure(figsize=(15, 8))
sns.boxplot(y=df["PH"], color="purple")
plt.xlabel("PH")
plt.ylabel("VALUE")
plt.title("Box Plot of PH After Outlier Capping")
plt.show()
```



```
In [60]: # STEP 13: VALUE COUNTS FOR TARGET COLUMN (QUALITY)
df["QUALITY"].value_counts()
```

```
Out[60]:
```

	count
--	-------

QUALITY	
5	483
6	462
7	143
4	33
8	16
3	6

dtype: int64

```
In [61]: # Unique values of every column
df.nunique()
```

Out[61]:

	0
FIXED_ACIDITY	91
VOLATILE_ACIDITY	135
CITRIC_ACID	77
RESIDUAL_SUGAR	80
CHLORIDES	131
FREE_SULFUR_DIOXIDE	53
TOTAL_SULFUR_DIOXIDE	138
DENSITY	388
PH	76
SULPHATES	89
ALCOHOL	61
QUALITY	6
ID	1143

dtype: int64

In [62]: `# STEP 14: RE CHECK`
`df.head()`

Out[62]:

	FIXED_ACIDITY	VOLATILE_ACIDITY	CITRIC_ACID	RESIDUAL_SUGAR	CHLORIDE
0	7.4	0.70	0.00	1.9	0.07
1	7.8	0.88	0.00	2.6	0.09
2	7.8	0.76	0.04	2.3	0.09
3	11.2	0.28	0.56	1.9	0.07
4	7.4	0.70	0.00	1.9	0.07

In []:

STEP-2: DEEP LEARNING (ANN)

In [63]: `# STEP 1: LOAD REQUIRED LIB`
`from sklearn.model_selection import train_test_split # for train-test split`
`from sklearn.metrics import mean_absolute_error, mean_squared_error # for c`
`from sklearn.ensemble import ExtraTreesRegressor # for feature importance`

```
In [64]: # STEP 2 : DEFINE X AND Y
X = df.drop("QUALITY", axis=1) # input features
y = df["QUALITY"]             # target column
```

```
In [65]: # STEP 3 : CORRELATION MATRIX
df.corr() # numerical correlation between columns
```

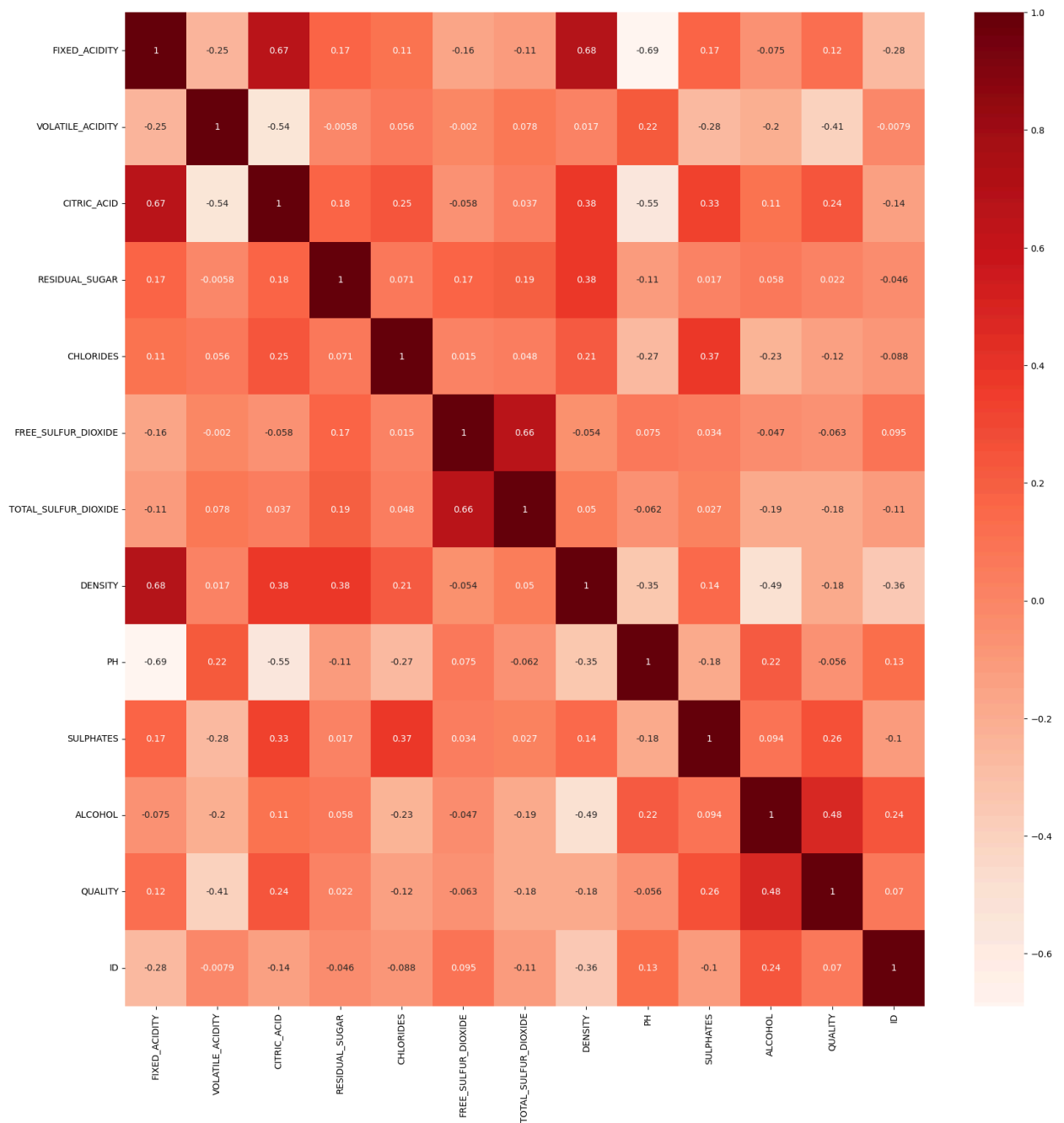
```
Out[65]:
```

	FIXED_ACIDITY	VOLATILE_ACIDITY	CITRIC_ACID	RESIDUAL_SUGAR
FIXED_ACIDITY	1.000000	-0.250728	0.673157	
VOLATILE_ACIDITY	-0.250728	1.000000	-0.544187	
CITRIC_ACID	0.673157	-0.544187	1.000000	
RESIDUAL_SUGAR	0.171831	-0.005751	0.175815	
CHLORIDES	0.107889	0.056336	0.245312	
FREE_SULFUR_DIOXIDE	-0.164831	-0.001962	-0.057589	
TOTAL_SULFUR_DIOXIDE	-0.110628	0.077748	0.036871	
DENSITY	0.681501	0.016512	0.375243	
PH	-0.689658	0.222534	-0.550071	
SULPHATES	0.174592	-0.276079	0.331232	
ALCOHOL	-0.075055	-0.203909	0.106250	
QUALITY	0.121970	-0.407394	0.240821	
ID	-0.275826	-0.007892	-0.139011	

```
In [66]: # STEP 4: HEATMAP OF CORRELATION MATRIX

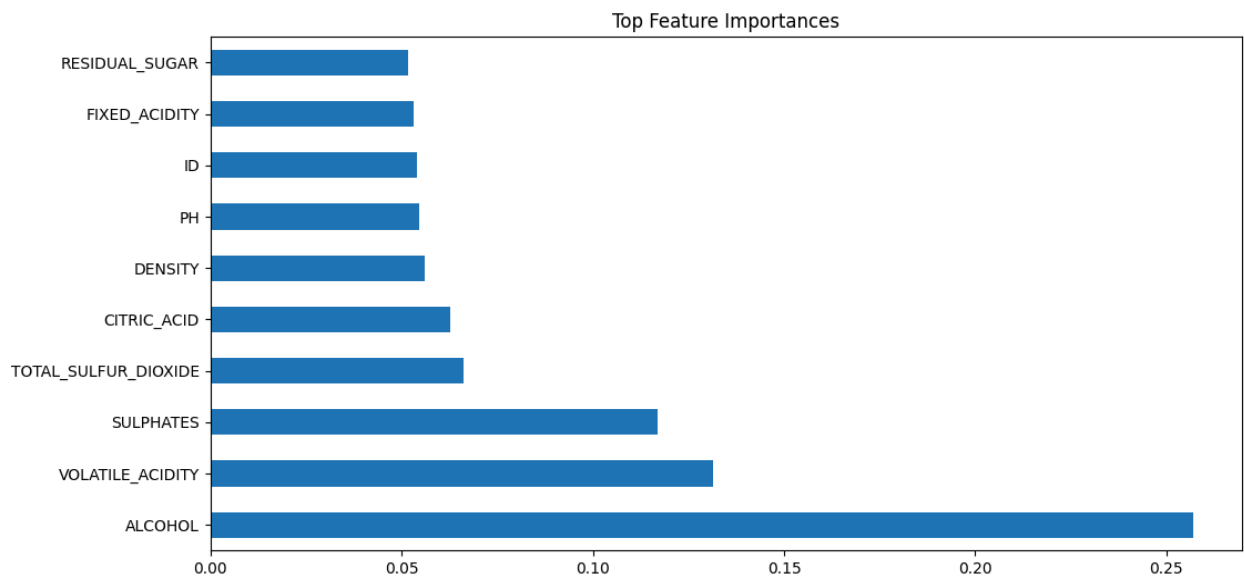
corrmat = df.corr() # calculate correlation
top_corr_features = corrmat.index # get column names

plt.figure(figsize=(20,20))
sns.heatmap(df[top_corr_features].corr(), annot=True, cmap="Reds")
plt.show()
```

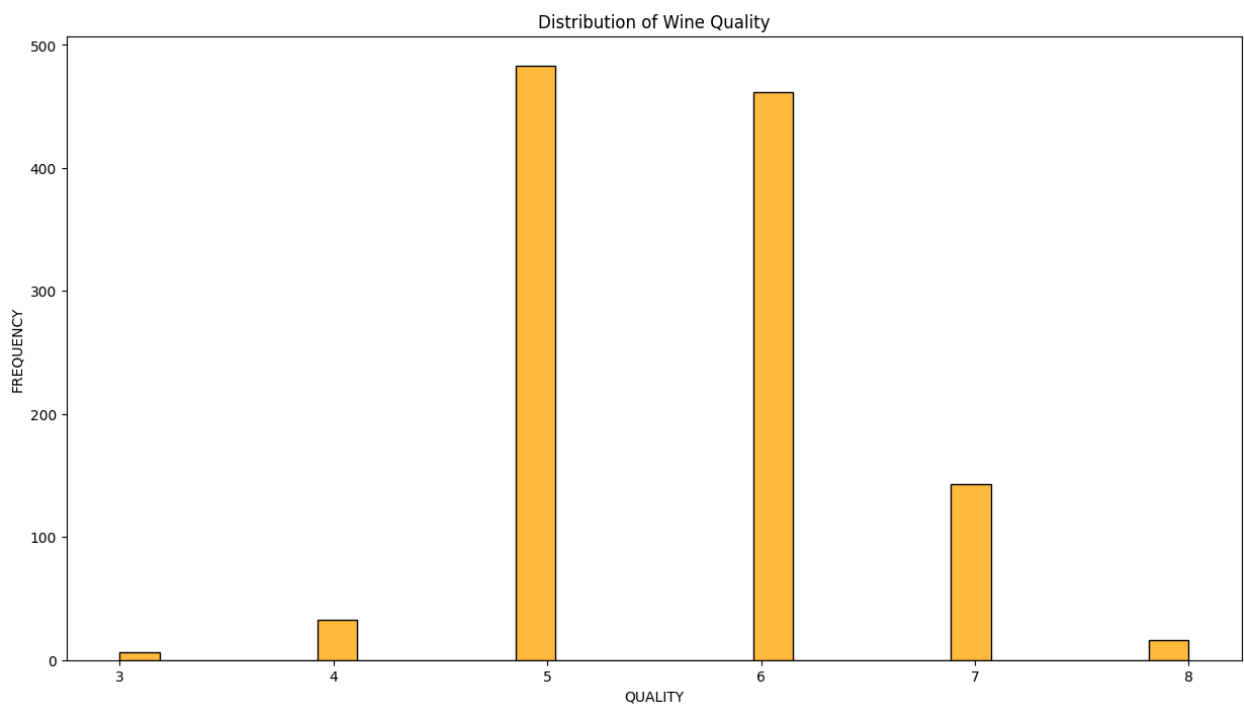



```
In [67]: # STEP 5 : FEATURE IMPORTANCE USING EXTRATREES
model = ExtraTreesRegressor()
model.fit(X, y)
plt.figure(figsize=(12,6))
feat_importances = pd.Series(model.feature_importances_, index=X.columns)
feat_importances.nlargest(10).plot(kind='barh') # show top important featur

plt.title("Top Feature Importances")
plt.show()
```



```
In [68]: # STEP 6: DISTRIBUTION OF TARGET VARIABLE
plt.figure(figsize=(15, 8))
sns.histplot(y, color="orange")
plt.xlabel("QUALITY")
plt.ylabel("FREQUENCY")
plt.title("Distribution of Wine Quality")
plt.show()
```



```
In [69]: # STEP 6 : DISTRIBUTION OF TARGET VARIABLE
plt.figure(figsize=(12,6))
sns.distplot(y)
plt.title("Distribution")
plt.show()
```

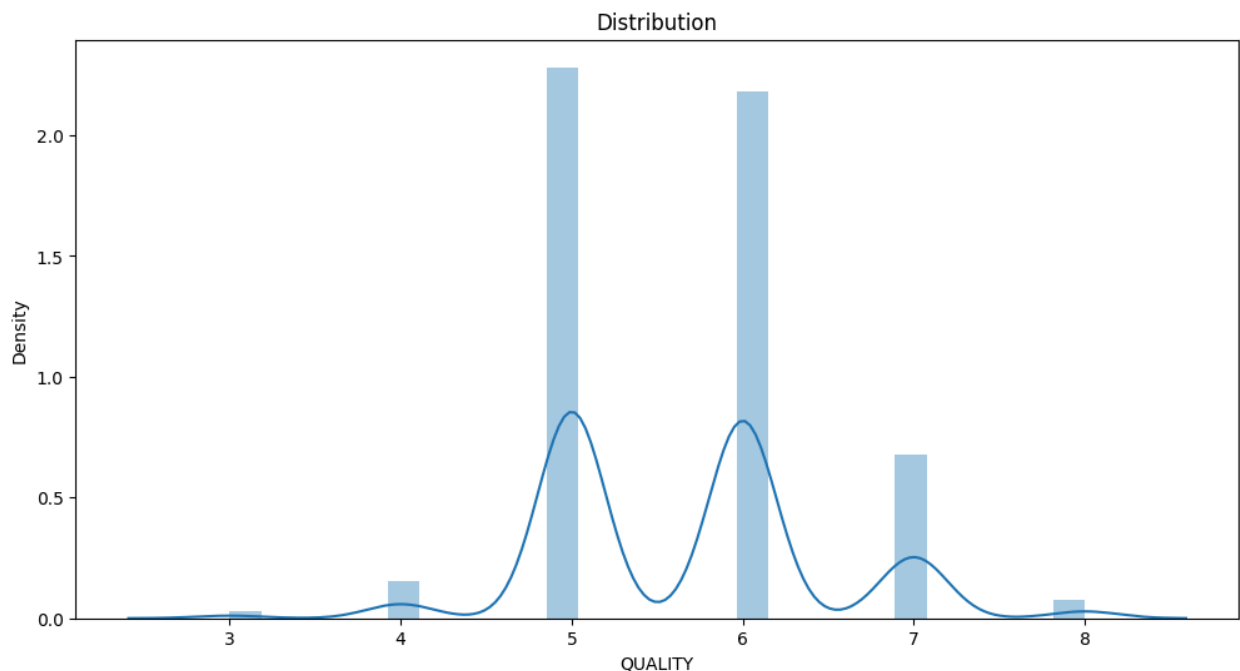
```
/tmp/ipython-input-999313654.py:3: UserWarning:
```

```
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(y)
```



```
In [70]: # STEP 7 : TRAIN-TEST SPLIT ( TRAIN= 70% AND TEST= 30%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
In [71]: # STEP 8: BUILD ANN MODEL
from keras.models import Sequential
from keras.layers import Dense, Dropout
from keras.layers import LeakyReLU, PReLU, ELU
```

```
In [72]: NN_model = Sequential() # create empty neural network
```

```
In [75]: # Input Layer
NN_model.add(Dense(64, input_dim=X_train.shape[1], activation="relu"))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [76]: # Hidden Layers
```

```

NN_model.add(Dense(128, kernel_initializer='normal', activation='relu')) # fi
NN_model.add(Dense(64, kernel_initializer='normal', activation='relu')) # sec

```

```

In [77]: # Output Layer
NN_model.add(Dense(1, kernel_initializer='normal', activation='linear'))

```

```

In [78]: # STEP 9: COMPILE THE MODEL
NN_model.compile(
    loss='mean_absolute_error',
    optimizer='adam',
    metrics=['mean_absolute_error'])

```

```

In [79]: NN_model.summary() # show full model structure

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	832
dense_2 (Dense)	(None, 64)	4,160
dense_3 (Dense)	(None, 128)	8,320
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 1)	65

Total params: 21,633 (84.50 KB)

Trainable params: 21,633 (84.50 KB)

Non-trainable params: 0 (0.00 B)

```

In [80]: # STEP 10: TRAIN THE MODEL
model_history = NN_model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=8)

```

Epoch 1/50
80/80  **3s** 9ms/step - loss: 2.7037 - mean_absolute_error: 2.7037 - val_loss: 1.4216 - val_mean_absolute_error: 1.4216

Epoch 2/50
80/80  **1s** 5ms/step - loss: 1.4488 - mean_absolute_error: 1.4488 - val_loss: 1.2110 - val_mean_absolute_error: 1.2110

Epoch 3/50
80/80  **1s** 7ms/step - loss: 1.1647 - mean_absolute_error: 1.1647 - val_loss: 0.7425 - val_mean_absolute_error: 0.7425

Epoch 4/50
80/80  **0s** 4ms/step - loss: 0.7384 - mean_absolute_error: 0.7384 - val_loss: 1.3062 - val_mean_absolute_error: 1.3062

Epoch 5/50
80/80  **0s** 3ms/step - loss: 0.8028 - mean_absolute_error: 0.8028 - val_loss: 0.6156 - val_mean_absolute_error: 0.6156

Epoch 6/50
80/80  **0s** 4ms/step - loss: 0.7357 - mean_absolute_error: 0.7357 - val_loss: 0.6889 - val_mean_absolute_error: 0.6889

Epoch 7/50
80/80  **0s** 4ms/step - loss: 0.7133 - mean_absolute_error: 0.7133 - val_loss: 0.5672 - val_mean_absolute_error: 0.5672

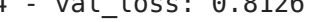
Epoch 8/50
80/80  **0s** 3ms/step - loss: 0.7661 - mean_absolute_error: 0.7661 - val_loss: 0.5836 - val_mean_absolute_error: 0.5836

Epoch 9/50
80/80  **0s** 4ms/step - loss: 0.7713 - mean_absolute_error: 0.7713 - val_loss: 0.6176 - val_mean_absolute_error: 0.6176

Epoch 10/50
80/80  **0s** 4ms/step - loss: 0.6638 - mean_absolute_error: 0.6638 - val_loss: 0.5994 - val_mean_absolute_error: 0.5994

Epoch 11/50
80/80  **0s** 4ms/step - loss: 0.7207 - mean_absolute_error: 0.7207 - val_loss: 0.6482 - val_mean_absolute_error: 0.6482

Epoch 12/50
80/80  **0s** 3ms/step - loss: 0.7089 - mean_absolute_error: 0.7089 - val_loss: 0.5507 - val_mean_absolute_error: 0.5507

Epoch 13/50
80/80  **0s** 4ms/step - loss: 0.6334 - mean_absolute_error: 0.6334 - val_loss: 0.8126 - val_mean_absolute_error: 0.8126

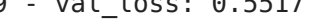
Epoch 14/50
80/80  **0s** 3ms/step - loss: 0.7268 - mean_absolute_error: 0.7268 - val_loss: 0.5929 - val_mean_absolute_error: 0.5929

Epoch 15/50
80/80  **0s** 3ms/step - loss: 0.6178 - mean_absolute_error: 0.6178 - val_loss: 0.6135 - val_mean_absolute_error: 0.6135

Epoch 16/50
80/80  **0s** 4ms/step - loss: 0.7265 - mean_absolute_error: 0.7265 - val_loss: 0.6115 - val_mean_absolute_error: 0.6115

Epoch 17/50
80/80  **0s** 4ms/step - loss: 0.5991 - mean_absolute_error: 0.5991 - val_loss: 0.6485 - val_mean_absolute_error: 0.6485

Epoch 18/50
80/80  **0s** 3ms/step - loss: 0.6020 - mean_absolute_error: 0.6020 - val_loss: 0.8071 - val_mean_absolute_error: 0.8071

Epoch 19/50
80/80  **0s** 3ms/step - loss: 0.7072 - mean_absolute_error: 0.7072 - val_loss: 0.6296 - val_mean_absolute_error: 0.6296
Epoch 20/50
80/80  **0s** 4ms/step - loss: 0.6580 - mean_absolute_error: 0.6580 - val_loss: 0.5901 - val_mean_absolute_error: 0.5901
Epoch 21/50
80/80  **0s** 3ms/step - loss: 0.8079 - mean_absolute_error: 0.8079 - val_loss: 0.8116 - val_mean_absolute_error: 0.8116
Epoch 22/50
80/80  **0s** 3ms/step - loss: 0.6987 - mean_absolute_error: 0.6987 - val_loss: 0.6272 - val_mean_absolute_error: 0.6272
Epoch 23/50
80/80  **0s** 4ms/step - loss: 0.6643 - mean_absolute_error: 0.6643 - val_loss: 0.5978 - val_mean_absolute_error: 0.5978
Epoch 24/50
80/80  **0s** 4ms/step - loss: 0.6312 - mean_absolute_error: 0.6312 - val_loss: 0.5350 - val_mean_absolute_error: 0.5350
Epoch 25/50
80/80  **0s** 3ms/step - loss: 0.6867 - mean_absolute_error: 0.6867 - val_loss: 0.5642 - val_mean_absolute_error: 0.5642
Epoch 26/50
80/80  **0s** 4ms/step - loss: 0.6147 - mean_absolute_error: 0.6147 - val_loss: 0.5734 - val_mean_absolute_error: 0.5734
Epoch 27/50
80/80  **0s** 3ms/step - loss: 0.6544 - mean_absolute_error: 0.6544 - val_loss: 0.6628 - val_mean_absolute_error: 0.6628
Epoch 28/50
80/80  **0s** 3ms/step - loss: 0.6106 - mean_absolute_error: 0.6106 - val_loss: 0.6175 - val_mean_absolute_error: 0.6175
Epoch 29/50
80/80  **0s** 3ms/step - loss: 0.6485 - mean_absolute_error: 0.6485 - val_loss: 0.5297 - val_mean_absolute_error: 0.5297
Epoch 30/50
80/80  **0s** 4ms/step - loss: 0.6236 - mean_absolute_error: 0.6236 - val_loss: 0.9286 - val_mean_absolute_error: 0.9286
Epoch 31/50
80/80  **1s** 3ms/step - loss: 0.7379 - mean_absolute_error: 0.7379 - val_loss: 0.5517 - val_mean_absolute_error: 0.5517
Epoch 32/50
80/80  **0s** 3ms/step - loss: 0.5957 - mean_absolute_error: 0.5957 - val_loss: 0.5610 - val_mean_absolute_error: 0.5610
Epoch 33/50
80/80  **0s** 3ms/step - loss: 0.6200 - mean_absolute_error: 0.6200 - val_loss: 0.5881 - val_mean_absolute_error: 0.5881
Epoch 34/50
80/80  **0s** 3ms/step - loss: 0.7056 - mean_absolute_error: 0.7056 - val_loss: 0.5926 - val_mean_absolute_error: 0.5926
Epoch 35/50
80/80  **0s** 3ms/step - loss: 0.6284 - mean_absolute_error: 0.6284 - val_loss: 0.6097 - val_mean_absolute_error: 0.6097
Epoch 36/50
80/80  **1s** 6ms/step - loss: 0.5661 - mean_absolute_error: 0.5661 - val_loss: 0.5359 - val_mean_absolute_error: 0.5359

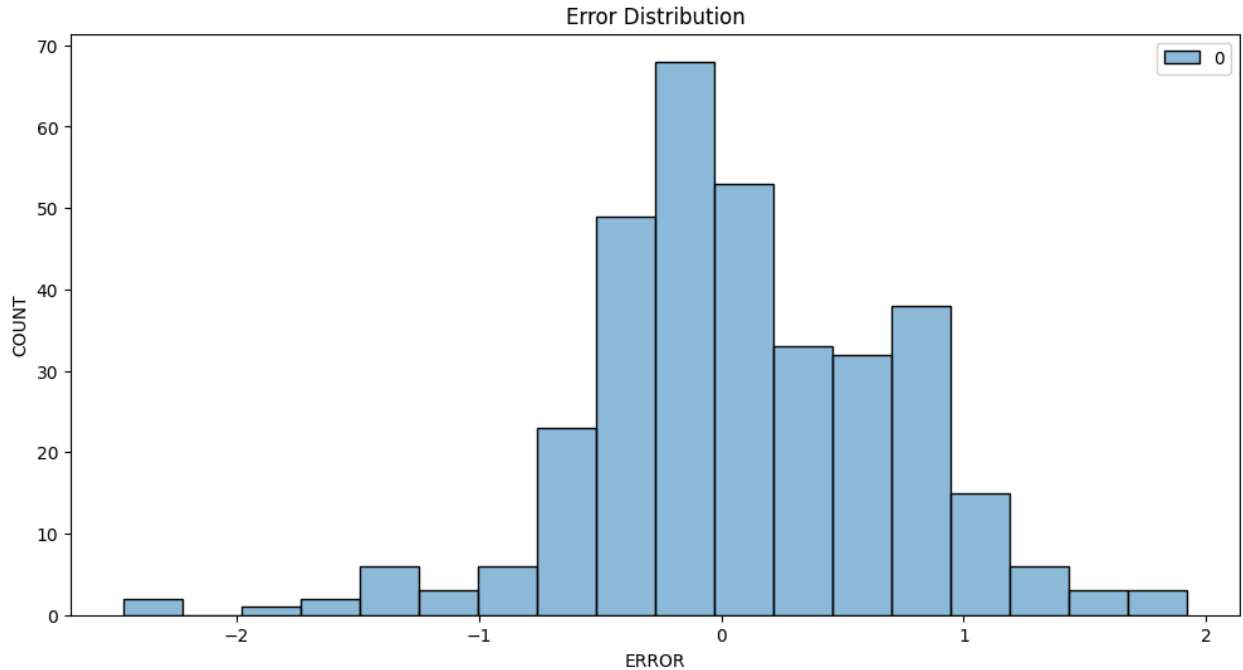
Epoch 37/50
80/80 ————— **1s** 5ms/step - loss: 0.6373 - mean_absolute_error: 0.6373 - val_loss: 0.7529 - val_mean_absolute_error: 0.7529
Epoch 38/50
80/80 ————— **0s** 5ms/step - loss: 0.6647 - mean_absolute_error: 0.6647 - val_loss: 0.5413 - val_mean_absolute_error: 0.5413
Epoch 39/50
80/80 ————— **0s** 5ms/step - loss: 0.6190 - mean_absolute_error: 0.6190 - val_loss: 0.7510 - val_mean_absolute_error: 0.7510
Epoch 40/50
80/80 ————— **1s** 6ms/step - loss: 0.6256 - mean_absolute_error: 0.6256 - val_loss: 0.5870 - val_mean_absolute_error: 0.5870
Epoch 41/50
80/80 ————— **0s** 3ms/step - loss: 0.5890 - mean_absolute_error: 0.5890 - val_loss: 0.5271 - val_mean_absolute_error: 0.5271
Epoch 42/50
80/80 ————— **0s** 3ms/step - loss: 0.6031 - mean_absolute_error: 0.6031 - val_loss: 0.7734 - val_mean_absolute_error: 0.7734
Epoch 43/50
80/80 ————— **0s** 4ms/step - loss: 0.6585 - mean_absolute_error: 0.6585 - val_loss: 0.5995 - val_mean_absolute_error: 0.5995
Epoch 44/50
80/80 ————— **0s** 3ms/step - loss: 0.5824 - mean_absolute_error: 0.5824 - val_loss: 0.6149 - val_mean_absolute_error: 0.6149
Epoch 45/50
80/80 ————— **0s** 3ms/step - loss: 0.6209 - mean_absolute_error: 0.6209 - val_loss: 0.5634 - val_mean_absolute_error: 0.5634
Epoch 46/50
80/80 ————— **0s** 3ms/step - loss: 0.6060 - mean_absolute_error: 0.6060 - val_loss: 0.5825 - val_mean_absolute_error: 0.5825
Epoch 47/50
80/80 ————— **0s** 3ms/step - loss: 0.6190 - mean_absolute_error: 0.6190 - val_loss: 0.5515 - val_mean_absolute_error: 0.5515
Epoch 48/50
80/80 ————— **0s** 3ms/step - loss: 0.6185 - mean_absolute_error: 0.6185 - val_loss: 0.7349 - val_mean_absolute_error: 0.7349
Epoch 49/50
80/80 ————— **0s** 3ms/step - loss: 0.6287 - mean_absolute_error: 0.6287 - val_loss: 0.5657 - val_mean_absolute_error: 0.5657
Epoch 50/50
80/80 ————— **0s** 3ms/step - loss: 0.5818 - mean_absolute_error: 0.5818 - val_loss: 0.5109 - val_mean_absolute_error: 0.5109

```
In [81]: # STEP 11 : PREDICTING
         prediction = NN_model.predict(X_test)
```

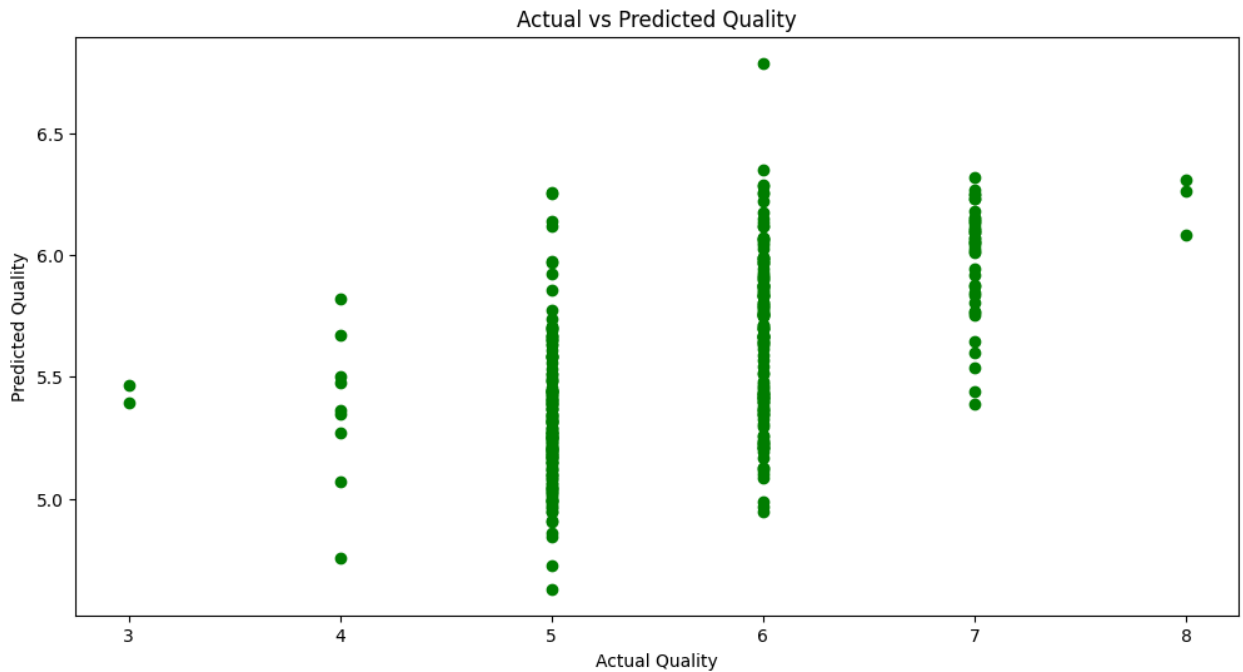
11/11 ————— **0s** 14ms/step

```
In [82]: # STEP 12: DISTRIBUTION OF ERRORS
         plt.figure(figsize=(12,6))
         sns.histplot(y_test.values.reshape(-1,1) - prediction, color="purple")
         plt.xlabel("ERROR")
         plt.ylabel("COUNT")
         plt.title("Error Distribution")
```

```
plt.show()
```



```
In [84]: # STEP 13: SCATTERPLOT OF ACTUAL VS PREDICTED
plt.figure(figsize=(12, 6))
plt.scatter(y_test, prediction, color="green")
plt.xlabel("Actual Quality")
plt.ylabel("Predicted Quality")
plt.title("Actual vs Predicted Quality")
plt.show()
```



```
In [85]: # STEP 14: EVALUATION METRICS
MAE = mean_absolute_error(y_test, prediction)
```



```
MSE = mean_squared_error(y_test, prediction)
RMSE = np.sqrt(MSE)

print("MAE:", MAE) # average absolute error
print("MSE:", MSE) # squared error
print("RMSE:", RMSE) # root mean squared error
```

MAE: 0.49368229508399963

MSE: 0.4154598116874695

RMSE: 0.6445617206191114

Final Conclusion

- ANN model successfully predicted wine quality.
- Training and validation loss reduced properly.
- Predicted values are close to actual values.
- Low MAE and RMSE indicate good accuracy.

In []: